spend&Track — Database Schema Design

PostgreSQL schema for the React + FastAPI expense tracker · 1 October 2026

Database	PostgreSQL 16 (compatible with 13+) · database name `spendai_db`
Tables	`users`, `expenses`, `budgets`
DDL script	`backend-fastapi/database/schema.sql` (same as `app/config/migrate.py`, idempotent)
Accessed by	`backend-fastapi/app/config/db.py` → `query()` (psycopg 3 async pool, 1–10 connections)

1. Entity-relationship diagram

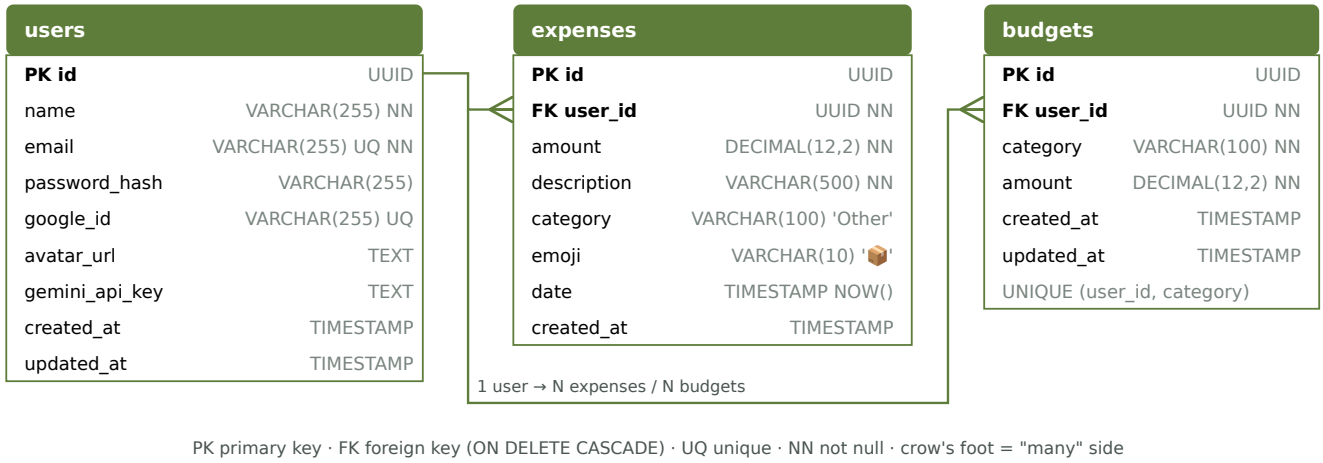


Figure 1 — users own expenses and budgets; deleting a user deletes all of their rows.

2. Table: users

One row per account. An account can sign in with email/password, Google, or both once Google is linked by email.

Column	Type	Null	Default	Key	Description
id	UUID	NO	gen_random_uuid()	PK	User ID (JWT claim `userId`)
name	VARCHAR(255)	NO	—		Display name (trimmed)
email	VARCHAR(255)	NO	—	UQ	Login email, stored lower-case
password_hash	VARCHAR(255)	YES	NULL		bcrypt ($2a$, cost 12); NULL means Google-only account
google_id	VARCHAR(255)	YES	NULL	UQ	Google sub claim
avatar_url	TEXT	YES	NULL		Google profile picture URL
gemini_api_key	TEXT	YES	NULL		Optional per-user Gemini key (not used by the current UI)
created_at	TIMESTAMP	YES	NOW()		Row creation time
updated_at	TIMESTAMP	YES	NOW()		Set to NOW() when the API key is updated

Rules enforced by the API (`auth_controller.py`)

- **Register:** name, email and password are required, the password needs at least 6 characters, and the email must be unique (409 if taken).
- **Login:** an account without `password_hash` gets "This account uses Google sign-in".
- **Google:** the email must be verified by Google. The API matches by `google_id`, then links by email, otherwise inserts a new user.
- `password_hash` is never returned to the client.

Used by

Endpoint	Operation
POST /api/auth/register	INSERT
POST /api/auth/login	SELECT by email
POST /api/auth/google	SELECT / UPDATE (link google_id, avatar) / INSERT
GET /api/auth/me + every protected request	SELECT by id (`middleware/auth.py`)
PATCH /api/auth/api-key	UPDATE gemini_api_key, updated_at

3. Table: expenses

Each money-spent entry logged by a user.

Column	Type	Null	Default	Key	Description
id	UUID	NO	gen_random_uuid()	PK	Expense ID
user_id	UUID	NO	—	FK	Owner → `users.id` (ON DELETE CASCADE)
amount	DECIMAL(12,2)	NO	—		Amount in ₹; the API requires > 0
description	VARCHAR(500)	NO	—		e.g. "Uber to office" (trimmed)
category	VARCHAR(100)	NO	'Other'		Food, Transport, Shopping, Health, Entertainment, Recharge, Education, Grocerries, Other
emoji	VARCHAR(10)	YES	'📦'		Category icon
date	TIMESTAMP	YES	NOW()		When the money was spent
created_at	TIMESTAMP	YES	NOW()		Row creation time

Rules enforced by the API (`expense_controller.py`)

- Amount and description are required, and the amount must be a positive number.
- Every query filters by `user_id` = the logged-in user, so users only ever see their own rows.
- Updating or deleting another user's expense returns 404.

Used by

Endpoint	Operation
GET /api/expenses	List, newest first. Filters: `month`, `category`; `limit` 100, `offset` 0
POST /api/expenses	INSERT
PUT /api/expenses/{id}	Partial UPDATE (COALESCE)
DELETE /api/expenses/{id}	DELETE
GET /api/expenses/stats/summary	Totals, by category, last 14 days, this month, list of months
GET /api/expenses/stats/daily	Per-day totals between `from` and `to`

4. Table: budgets

A monthly spending limit per category, per user.

Column	Type	Null	Default	Key	Description
id	UUID	NO	gen_random_uuid()	PK	Budget ID
user_id	UUID	NO	—	FK, UQ*	Owner → `users.id` (ON DELETE CASCADE)
category	VARCHAR(100)	NO	—	UQ*	Category the limit applies to
amount	DECIMAL(12,2)	NO	—		Limit in ₹; the API requires > 0
created_at	TIMESTAMP	YES	NOW()		Row creation time
updated_at	TIMESTAMP	YES	NOW()		Set to NOW() on every upsert

* `UNIQUE (user_id, category)`: a user has at most one budget per category.

Rules and usage

- Category and amount are required, and the amount must be > 0 (`budget_controller.py`).
- **POST /api/budgets** is an upsert: `INSERT … ON CONFLICT (user_id, category) DO UPDATE`.
- **GET /api/budgets** lists budgets ordered by category. **DELETE /api/budgets/{category}** removes one.
- Alerts are computed in the frontend (`Dashboard.js`): spent ÷ budget ≥ 100% shows "exceeded", ≥ 80% shows "x% — ₹y left".

5. Constraints & indexes

Name	Type	Table	Columns	Why
users_pkey	PK	users	id	
users_email_key	UNIQUE	users	email	One account per email
users_google_id_key	UNIQUE	users	google_id	One account per Google user
expenses_pkey	PK	expenses	id	
expenses_user_id_fkey	FK CASCADE	expenses	user_id → users.id	Deleting a user deletes their expenses
budgets_pkey	PK	budgets	id	
budgets_user_id_fkey	FK CASCADE	budgets	user_id → users.id	Deleting a user deletes their budgets
budgets_user_id_category_key	UNIQUE	budgets	(user_id, category)	Upsert target
idx_expenses_user_id	INDEX	expenses	user_id	Every expense query filters by user
idx_expenses_date	INDEX	expenses	date	Sorting and date-range stats
idx_budgets_user_id	INDEX	budgets	user_id	Budget list per user

6. Key queries

Month filter

```
WHERE user_id = $1 AND TO_CHAR(date, 'YYYY-MM') = $2
```

Stats totals

```
SELECT COUNT(*) AS count, COALESCE(SUM(amount),0) AS total,
       COUNT(DISTINCT date::date) AS active_days
FROM expenses WHERE user_id = $1 [AND month filter]
```

By category

```
SELECT category, MIN(emoji) AS emoji, COALESCE(SUM(amount),0) AS total, COUNT(*) AS count
FROM expenses WHERE user_id = $1 [AND month filter]
GROUP BY category ORDER BY total DESC
```

Daily totals (trend chart)

```
SELECT TO_CHAR(date,'YYYY-MM-DD') AS day, SUM(amount) AS total
FROM expenses
WHERE user_id = $1 AND date >= $2::date AND date < $3::date + INTERVAL '1 day'
GROUP BY day ORDER BY day
```

Budget upsert

```
INSERT INTO budgets (user_id, category, amount) VALUES ($1, $2, $3)
ON CONFLICT (user_id, category) DO UPDATE SET amount = $3, updated_at = NOW()
RETURNING *
```

7. Design notes

- **UUID primary keys:** IDs can't be guessed, and Postgres generates them (`gen_random_uuid()`).
- **DECIMAL(12,2) for money:** values are exact, with no floating-point rounding.
- **Category is free text** (VARCHAR), not a lookup table. The category list lives in the frontend (`Dashboard.js`, `utils/gemini.js`). "Grocerries" is the spelling stored in the data.
- **Timestamps have no time zone:** month/day grouping uses the database server's timezone.
- **ON DELETE CASCADE:** removing a user removes all of their expenses and budgets.
- **Guest mode** stores nothing in this database; its data lives in the browser's sessionStorage.
- **API value format:** NUMERIC/COUNT values come back as strings and timestamps as ISO-8601 UTC strings (`config/db.py`), which the frontend expects.

8. Full DDL script

From `backend-fastapi/database/schema.sql`. Run with `psql "$DATABASE_URL" -f backend-fastapi/database/schema.sql`.

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;

BEGIN;

CREATE TABLE IF NOT EXISTS users (
  id             UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name           VARCHAR(255) NOT NULL,
  email          VARCHAR(255) UNIQUE NOT NULL,
  password_hash  VARCHAR(255),
  google_id      VARCHAR(255) UNIQUE,
  avatar_url     TEXT,
  gemini_api_key TEXT,
  created_at     TIMESTAMP DEFAULT NOW(),
  updated_at     TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS expenses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  amount      DECIMAL(12,2) NOT NULL,
  description VARCHAR(500) NOT NULL,
  category    VARCHAR(100) NOT NULL DEFAULT 'Other',
  emoji       VARCHAR(10) DEFAULT '📦',
  date        TIMESTAMP DEFAULT NOW(),
  created_at  TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS budgets (
  id         UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  category   VARCHAR(100) NOT NULL,
  amount     DECIMAL(12,2) NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  updated_at TIMESTAMP DEFAULT NOW(),
  UNIQUE (user_id, category)
);

CREATE INDEX IF NOT EXISTS idx_expenses_user_id ON expenses(user_id);
CREATE INDEX IF NOT EXISTS idx_expenses_date    ON expenses(date);
CREATE INDEX IF NOT EXISTS idx_budgets_user_id  ON budgets(user_id);
```

```
COMMIT;
```